VendorHub SRS Document

Table of Contents

Section	Title	Page
1	Introduction	2
1.1	Purpose	2
1.2	Scope	2
1.3	Definitions and Acronyms	3
2	System Overview	4
2.1	What VendorHub Does	4
2.2	User Roles	4
2.3	High-Level Architecture	5
3	Functional Requirements	5
3.1	FR-1 Authentication & Authorization	5
3.2	FR-2 Vendor Management	6
3.3	FR-3 Product Management	6
3.4	FR-4 Shopping & Checkout	6
3.5	FR-5 Order Management	7
3.6	FR-6 Notifications	7
4	Non-Functional Requirements	7
4.1	Performance	7

Section	Title	Page
4.2	Scalability	8
4.3	Security	8
4.4	Availability	8
4.5	Reliability & Maintainability	9
5	API Design (Spring Boot REST Endpoints)	9
5.1	Auth Service APIs	9
5.2	Product Service APIs	10
5.3	Order Service APIs	10
5.4	Notification Flow (Kafka-based)	10
6	Object Constraint Language (OCL)	10
6.1	User Constraints	10
6.2	Product Constraints	11
6.3	Cart Constraints	12
6.4	Product Category Constraints	14
6.5	Category Constraints	14
6.6	Business Rule Constraint (Checkout Logic)	15
6.7	Order Constraints	15
6.8	Notification Constraints	16
6.9	Role Constraints	16
7	Design Patterns and Aspect-Oriented Programming (AOP)	17
7.1	Design Patterns Used in VendorHub	17

Section	Title	Page
7.1.1	Repository Pattern	17
7.1.2	Service Layer Pattern	18
7.1.3	DTO Pattern	19
7.1.4	Observer Pattern	20
7.1.5	Singleton Pattern	21
7.1.6	Gateway Filter Pattern	22
7.2	Aspect-Oriented Programming (AOP)	23
7.2.1	Logging Aspect	23
7.2.2	Exception Handling Aspect	24
7.2.3	Security Cross-Cutting Concern	25
7.2.4	Benefits of AOP in VendorHub	25
8	Dockerization and Deployment	26
8.1	Docker Architecture	26
8.2	Docker Compose Orchestration	27
8.3	Database Containerization	28
9	Microservices and Cloud Architecture	28
9.1	Microservices Overview	28
9.2	Eureka Service Discovery	29
9.3	Spring Cloud Config Server	29
9.4	API Gateway Architecture	30
9.5	Event-Driven Communication Using Kafka	30
9.6	Database-per-Service Pattern	31

Section	Title	Page
9.7	Scalability in VendorHub	31
9.8	Fault Isolation and Reliability	32
10	Clean Code Principles	32
10.1	Separation of Concerns	32
10.2	Layered Architecture	33
10.3	Meaningful Naming Conventions	33
10.4	DTO Usage	34
10.5	Global Exception Handling	34
10.6	Validation and Input Checking	35
10.7	Logging and Monitoring	35
10.8	Modularity	35
11	Security Implementation	36
11.1	JWT Authentication	36
11.2	API Gateway Security	37
11.3	Role-Based Authorization	37
11.4	Password Encryption	37
11.5	Validation Security	38
11.6	Stateless Session Management	38
11.7	Exception Security Handling	38
12	Database Design	38
12.1	Main Entities	39
12.2	User Entity	39

Section	Title	Page
12.3	Product Entity	40
12.4	Order Management Entities	40
12.5	Notification Entity	41
12.6	Database Relationships	41
13	System Diagrams	42
13.1	Use Case Diagrams	42
13.2	Activity Diagrams	43
13.3	Sequence Diagrams	47
13.4	Class Diagrams	53
13.5	Entity Relationship Diagram (ERD)	57
14	Conclusion	61

1. Introduction

1.1 Purpose

VendorHub is a microservices-based e-commerce web application that connects administrators, vendors, and customers through a centralized platform. The system enables vendors to manage products and track sales, customers to browse and purchase products, and administrators to manage vendors and platform activities.

This course project demonstrates the use of modern software engineering technologies and concepts including RESTful APIs, Aspect-Oriented Programming (AOP), Docker, microservices, Spring Cloud, and secure authentication using Spring Boot and React.

1.2 Scope

VendorHub is a web-based e-commerce system developed using a microservices architecture. The system provides the following features:

- User registration and authentication
- Role-based access for admins, vendors, and customers
- Vendor and product management
- Product browsing and shopping cart management
- Favorites and checkout functionality
- Vendor sales dashboard and analytics
- Notification services using Apache Kafka
- REST API communication between services
- Dockerized deployment and cloud support

The system consists of the following services:

- Eureka Server
- Config Server
- API Gateway
- Auth Service
- Product Service

- Order Service
- Notification Service

1.3 Definitions and Acronyms

Acronym	Definition
API	Application Programming Interface
REST	Representational State Transfer
AOP	Aspect-Oriented Programming
JWT	JSON Web Token
Kafka	Messaging and event-streaming platform
Docker	Containerization platform
RBAC	Role-Based Access Control
CRUD	Create, Read, Update, Delete
Spring Boot	Backend development framework
Spring Cloud	Framework for microservices
React	Frontend JavaScript library
API Gateway	Service routing entry point
Eureka Server	Service discovery server
Config Server	Centralized configuration service
Vendor	Seller managing products
Customer	User purchasing products
Admin	System administrator

2. System Overview

2.1 What VendorHub Does

VendorHub provides an end-to-end e-commerce workflow where:

- Customers browse products, manage carts and favorites, and complete checkout by selecting a payment method and placing orders.
- Vendors manage their own product catalogs and monitor sales performance through a dashboard.
- Administrators oversee vendors and control product approval and deletion to maintain platform quality.

Order placement is fully functional, and once confirmed, sales data is automatically updated for the corresponding vendor.

2.2 User Roles

Admin

- Manage vendors (approve/reject)
- Approve or delete products
- Oversee platform operations

Vendor

- Add new products
- Edit products (price, stock quantity, details)
- Delete products
- View and manage own sales dashboard

Customer

- Browse products
- Add products to cart or favorites
- Proceed to checkout and place orders

2.3 High-Level Architecture

VendorHub follows a **microservices architecture** using Spring Boot and Spring Cloud.

Core Components:

- **Frontend:** React
- **API Gateway:** Central request routing
- **Eureka Server:** Service discovery
- **Config Server:** Centralized configuration
- **Auth Service:** Authentication & JWT security
- **Product Service:** Product management
- **Order Service:** Order processing & checkout
- **Notification Service:** Kafka-based event notifications

---

3. Functional Requirements

FR-1 Authentication & Authorization

- The system shall allow users to register using email, password, and role (Customer or Vendor).
- The system shall ensure email uniqueness during registration.
- The system shall allow users to log in using valid credentials.
- The system shall authenticate users and provide secure session access using tokens.
- The system shall enforce role-based access control (RBAC) for protected resources.
- The system shall allow administrators to approve or reject vendor registration requests.
- The system shall lock user accounts after multiple failed login attempts for security purposes.

FR-2 Vendor Management

- The system shall allow administrators to review vendor registration requests.
- The system shall allow approved vendors to manage their profile and business information.
- The system shall notify vendors regarding approval or rejection of their application.

FR-3 Product Management

- The system shall allow vendors to create, update, and delete their products.
- The system shall require new products to be approved before becoming visible to customers.
- The system shall allow administrators to approve or reject product listings.
- The system shall prevent customers from accessing unapproved products.
- The system shall allow vendors to update product price and stock.

FR-4 Shopping & Checkout

- The system shall allow customers to browse available products with filtering and sorting options.
- The system shall allow customers to manage a shopping cart (add/remove products).
- The system shall allow customers to manage a wishlist or favorites list.
- The system shall calculate total order cost including applicable charges.
- The system shall allow customers to complete checkout using available payment methods.
- The system shall update product stock after successful order placement.
- The system shall generate an order upon successful checkout.

FR-5 Order Management

- The system shall store and maintain order history for customers.
- The system shall allow customers to track their order status.
- The system shall allow vendors to view orders related to their products.

FR-6 Notifications

- The system shall notify users about key system events such as order placement, vendor approval, and stock updates.
- The system shall support email-based notifications for important events.

---

4. Non-Functional Requirements

4.1 Performance

- The system shall respond to 95% of user requests within 500ms under normal operating load.
- The system shall provide product search and browsing functionality with response times under 300ms.
- The system shall ensure frontend page load times remain under 2 seconds under normal network conditions.

4.2 Scalability

- The system shall support horizontal scaling of microservices independently based on demand.
- The system shall support a growing number of concurrent users without degradation of core functionality.
- The system shall support high-throughput event processing for asynchronous communication between services.

4.3 Security

- All communication between clients and services shall be secured using HTTPS.
- The system shall implement token-based authentication with time-limited session validity.
- User passwords shall be securely stored using strong hashing algorithms.
- The system shall implement role-based access control (RBAC) for protected resources.
- The system shall comply with standard web security practices and mitigate common security vulnerabilities (e.g., OWASP Top 10).

4.4 Availability

- The system shall be designed to achieve high availability (target of 99% uptime).
- The failure of an individual microservice shall not affect the availability of the entire system.
- Critical services shall be designed with fault tolerance and recovery mechanisms.

4.5 Reliability & Maintainability

- The system shall ensure data consistency through transactional database operations where applicable.
- The system shall support reliable message delivery for asynchronous communication between services.
- The system shall be designed to support maintainability through modular microservices architecture.
- The system shall maintain adequate automated test coverage for core business functionality.

5. API Design (Spring Boot REST Endpoints)

VendorHub exposes RESTful APIs through a microservices architecture. All services communicate via JSON over HTTP and are accessed through an API Gateway.

Authentication is handled using JWT-based security, and role-based access control (RBAC) is enforced across all endpoints.

5.1 Auth Service APIs Examples:

Method	Endpoint	Description	Role
POST	/api/auth/register	Register new user (customer/vendor)	Public
GET	/api/admin/users	Get all users	Admin
PUT	/api/admin/vendors/approve/{id}	Approve vendor	Admin

5.2 Product Service APIs Examples:

Method	Endpoint	Description	Role
POST	/api/products	Add product	Vendor
PUT	/api/products/{id}	Update product (price, stock, details)	Vendor
DELETE	/api/products/{id}	Delete product	Vendor/Admin
GET	/api/products/sort?type=newest	Sort products (newest, price, etc.)	Public

---

5.3 Order Service APIs Examples:

Method	Endpoint	Description	Role
POST	/api/cart/add	Add item to cart	Customer
GET	/api/cart	View cart	Customer
DELETE	/api/cart/remove/{id}	Remove item from cart	Customer

5.4 Notification Flow (Kafka-based)

Events are published asynchronously using Apache Kafka:

- OrderPlacedEvent
- ProductApprovedEvent
- ProductOutOfStockEvent
- UserRegisteredEvent
- VendorApprovedEvent

Notification Service consumes these events and triggers email/notification delivery.

---

6. Object Constraint Language (OCL)

OCL is used to define system-level business constraints over the UML model entities.

OCL Constraints Examples:

6.1 User Constraints

```
context User

inv ValidEmailFormat:

  self.email.matches('[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}')

inv StrongPassword:

  self.password.size() >= 8
```

```
inv VendorMustBeApproved:
  self.role = Role::VENDOR implies self.approved = true
inv ActiveRequiresApproval:
  self.status = UserStatus::ACTIVE implies self.approved = true
inv UniqueEmail:
  User.allInstances()->forAll(u1, u2 | u1 <> u2 implies u1.email <> u2.email)
pre CannotLoginIfInactive:
  self.status <> UserStatus::SUSPENDED and self.status <> UserStatus::DELETED
post UpdateStatusAfterApproval:
  self.approved = true implies self.status = UserStatus::ACTIVE
```

6.2 Product Constraints

```
context Product
inv PositivePrice:
  self.price > 0.0
inv NonNegativeStock:
  self.stock >= 0
inv VendorExistsAndApproved:
  self.vendor <> null and self.vendor.role = Role::VENDOR and self.vendor.approved
inv ApprovedStatusRequiresVendorApproval:
  self.status = ProductStatus::APPROVED implies self.vendor.approved
pre CannotPublishIfVendorNotApproved:
  self.status = ProductStatus::PENDING implies self.vendor.approved
post StockDecreaseOnSale:
self.stock = self.stock@pre - self.orderItems->sum(quantity)
```

6.3 Cart Constraints

```
context Cart

inv OneActiveCartPerUser:

  Cart.allInstances()->select(customer = self.customer and status = CartStatus::ACTIVE)-
>size() <= 1

inv CartBelongsToActiveUser:

  self.customer <> null and self.customer.status = UserStatus::ACTIVE

inv TotalAmountNonNegative:

  self.totalAmount >= 0.0

inv CartItemsNotEmpty:

  self.cartItems->notEmpty() implies self.cartItems->forAll(item | item <> null)

inv UniqueProductInCart:

  self.cartItems->forAll(p1, p2 | p1 <> p2 implies p1.product <> p2.product)

inv ValidQuantityInCartItems:

  self.cartItems->forAll(item | item.quantity > 0)

inv StockAvailabilityForActiveCart:

  self.status = CartStatus::ACTIVE implies

    self.cartItems->forAll(item | item.quantity <= item.product.stock)

inv PriceCalculationAccuracy:

  self.totalAmount = self.cartItems->sum(item | item.product.price * item.quantity)

pre CannotAddDuplicateProduct(product: Product):

  not self.cartItems->exists(item | item.product = product)

pre CannotAddIfProductInactive(product: Product):

  product.status = ProductStatus::APPROVED and product.vendor.approved = true

pre CannotAddIfQuantityExceedsStock(product: Product, quantity: Integer):

  quantity <= product.stock
```

```
pre CannotAddIfVendorNotApproved(product: Product):
  product.vendor.role = Role::VENDOR and product.vendor.approved = true
post AddItemToCart(product: Product, quantity: Integer):
  self.cartItems->exists(item | item.product = product and item.quantity = quantity)
post UpdateTotalAmountAfterAdd:
  self.totalAmount = self.totalAmount@pre + (product.price * quantity)
post RemoveItemFromCart(product: Product):
  not self.cartItems->exists(item | item.product = product)
post ClearCart:
  self.cartItems->isEmpty() and self.totalAmount = 0.0
```

6.4 Category Constraints

```
context Product
inv MustBelongToCategory:
self.category <> null
```

6.5 Category Constraints

```
context Category
inv NameNotEmpty:
  self.name <> null and self.name.size() > 0
inv NameMaxLength:
  self.name.size() <= 100
inv DescriptionMaxLength:
  self.description <> null implies self.description.size() <= 500
inv UniqueCategoryName:
  Category.allInstances()->forAll(c1, c2 |
    c1 <> c2 and c1.parent = c2.parent implies c1.name <> c2.name)
```

```
inv ValidParentReference:
  self.parent = null or (self.parent <> null and self.parent <> self)
inv NoCircularReference:
  not self.allParents()->includes(self)
inv ParentMustBeActive:
  self.parent <> null implies self.parent.isActive = true
inv ActiveIfParentActive:
  self.parent <> null and self.parent.isActive = true implies self.isActive = true
inv SlugFormat:
  self.slug.matches('[a-z0-9]+(-[a-z0-9]+)*')
inv UniqueSlug:
  Category.allInstances()->forAll(c1, c2 | c1 <> c2 implies c1.slug <> c2.slug)
pre CannotDeactivateIfHasActiveProducts:
  not self.products->exists(p | p.status = ProductStatus::APPROVED)
pre CannotDeleteIfHasSubcategories:
  self.subcategories->isEmpty()
pre CannotDeleteIfHasProducts:
  self.products->isEmpty()
post AddSubcategory(subcategory: Category):
  subcategory.parent = self
post RemoveSubcategory(subcategory: Category):
  subcategory.parent = null
post ActivateCategory:
  self.isActive = true
post DeactivateCategory:
```

```
  self.isActive = false and self.products->forAll(p | p.status <> ProductStatus::APPROVED)
```

6.6 Business Rule Constraint (Checkout Logic)

```
context Order
inv ValidCheckout:
self.status = 'PLACED' implies self.paymentMethod <> null
```

6.7 Order Constraint

```
context Order

inv PositiveTotalAmount:

  self.totalAmount > 0.0

inv NotEmptyItems:

  self.orderItems->notEmpty()

inv CustomerMustBeActive:

  self.customer <> null and self.customer.status = UserStatus::ACTIVE

pre ValidStatusTransition(newStatus: OrderStatus):

  (newStatus = OrderStatus::CONFIRMED and self.status@pre = OrderStatus::PENDING) or

  (newStatus = OrderStatus::SHIPPED and self.status@pre = OrderStatus::CONFIRMED) or

  (newStatus = OrderStatus::DELIVERED and self.status@pre = OrderStatus::SHIPPED) or

  (newStatus = OrderStatus::CANCELLED and self.status@pre <> OrderStatus::DELIVERED)

post CalculateTotalAmount:

  self.totalAmount = self.orderItems->sum(price * quantity)
```

6.8 Notification Constraint

```
context Notification

inv ValidTimestamp:

  self.createdAt <= OclDateTime::now()
```

```
inv MessageNotEmpty:
  self.message <> null and self.message.size() > 0
inv TypeMustBeValid:
  self.type.oclIsKindOf(NotificationType)
post MarkAsRead:
  self.isRead = true implies self.readAt <> null
```

6.9 Role Constraint

```
context Role
inv ValidRoleValues:
  self = Role::CUSTOMER or self = Role::VENDOR or self = Role::ADMIN
context UserStatus
inv ValidStatusValues:
  self = UserStatus::PENDING_EMAIL_VERIFICATION or
  self = UserStatus::EMAIL_VERIFIED or
  self = UserStatus::PENDING_APPROVAL or
  self = UserStatus::ACTIVE or
  self = UserStatus::SUSPENDED or
  self = UserStatus::DELETED
```

---

7. Design Patterns and Aspect-Oriented Programming (AOP)

7.1 Design Patterns Used in VendorHub

VendorHub applies several software design patterns to improve maintainability, scalability, modularity, and separation of concerns within the microservices architecture.

7.1.1 Repository Pattern

The Repository Pattern is used to abstract database access operations from the business logic layer. Spring Data JPA repositories provide reusable CRUD operations and custom database queries.

Example: UserRepository

```
@Repository
public interface UserRepository extends JpaRepository<User, Long> {

   Optional<User> findByEmail(String email);

   boolean existsByEmail(String email);

   List<User> findByRole(User.Role role);
}
```

Example: ProductRepository

```
@Query("SELECT p FROM Product p WHERE p.approvalStatus = 'APPROVED'
AND p.active = true AND p.price BETWEEN :min AND :max")
Page<Product> findApprovedByPriceRange(
   @Param("min") BigDecimal min,
   @Param("max") BigDecimal max,
   Pageable pageable);
```

Purpose

- Separates persistence logic from business logic
- Simplifies database interactions
- Reduces repetitive CRUD code
- Improves maintainability and scalability

7.1.2 Service Layer Pattern

The Service Layer Pattern centralizes business logic inside service classes and keeps controllers lightweight.

Example: ProductService

```
@Service
@RequiredArgsConstructor
@Slf4j
@Transactional
public class ProductService {

  private final ProductRepository productRepository;

  public ProductResponse createProduct(
      Long vendorId,
      String vendorEmail,
      CreateProductRequest req) {

    Product product = Product.builder()
      .name(req.getName())
      .price(req.getPrice())
      .vendorId(vendorId)
      .vendorEmail(vendorEmail)
      .approvalStatus(Product.ApprovalStatus.PENDING)
      .build();

    return ProductResponse.from(productRepository.save(product));
  }
}
```

Purpose

- Encapsulates business rules
- Improves modularity
- Facilitates testing and maintenance
- Separates application layers clearly

7.1.3 DTO (Data Transfer Object) Pattern

DTOs are used to transfer data between the backend and frontend without exposing database entities directly.

Example: AuthDTOs

```
@Data
public static class LoginRequest {

  @NotBlank @Email
  private String email;

  @NotBlank
  private String password;
}
```

Example: UserDTO

```
@Data
public static class UserDTO {

  private Long id;
  private String email;
  private String firstName;
  private User.Role role;

  public static UserDTO from(User user) {
    UserDTO dto = new UserDTO();
    dto.setId(user.getId());
    dto.setEmail(user.getEmail());
    dto.setRole(user.getRole());
    return dto;
  }
}
```

Purpose

- Prevents direct entity exposure
- Improves API security
- Controls serialized response structure

7.1.4 Observer Pattern (Event-Driven Architecture)

The Observer Pattern is implemented using Apache Kafka for asynchronous communication between microservices.

Services publish events while the Notification Service subscribes and reacts to them independently.

Example: Kafka Producer

```
@Service
@RequiredArgsConstructor
@Slf4j
public class OrderEventProducer {

  private final KafkaTemplate<String, Object> kafkaTemplate;

  public void publishOrderPlaced(OrderPlacedEvent event) {
    kafkaTemplate.send("order-placed",
        event.getOrderNumber(),
        event);
  }
}
```

Example: Kafka Consumer

```
@KafkaListener(
  topics = "order-placed",
  groupId = "notification-group"
)
public void onOrderPlaced(OrderPlacedEvent event) {

  notificationService.handleOrderPlaced(
    vendorId,
    vendorEmail,
    event.getOrderNumber(),
    event.getCustomerEmail(),
    productSummary,
```

```
        totalQty,
        vendorSubtotal,
        event.getOrderId()
    );
}
```

Purpose

- Enables loose coupling between services
- Supports asynchronous processing
- Improves scalability and fault tolerance
- Simplifies inter-service communication

7.1.5 Singleton Pattern

Spring Boot manages service and configuration classes as singleton beans by default.

Example

```
@Service
@RequiredArgsConstructor
public class AuthService {
}
```

Example

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
}
```

Purpose

- Ensures a single shared instance
- Reduces memory consumption
- Centralizes configuration and service logic

7.1.6 Gateway Filter Pattern

VendorHub uses a centralized API Gateway filter for authentication and authorization across all microservices.

Example: AuthenticationFilter

```
@Component
@RequiredArgsConstructor
@Slf4j
public class AuthenticationFilter implements GlobalFilter, Ordered {

  @Override
  public Mono<Void> filter(
      ServerWebExchange exchange,
      GatewayFilterChain chain) {

    String authHeader = exchange.getRequest()
        .getHeaders()
        .getFirst(HttpHeaders.AUTHORIZATION);

    if (authHeader == null ||
      !authHeader.startsWith("Bearer ")) {

      return reject(exchange, HttpStatus.UNAUTHORIZED);
    }

    return chain.filter(exchange);
  }
}
```

Purpose

- Centralizes authentication logic
- Secures all microservices consistently
- Reduces duplicated security code

7.2 Aspect-Oriented Programming (AOP)

VendorHub uses Aspect-Oriented Programming (AOP) to separate cross-cutting concerns such as logging, monitoring, and exception handling from business logic.

7.2.1 Logging Aspect

The application implements a logging aspect to monitor service and controller execution.

Example: LoggingAspect

```
@Aspect
@Component
@Slf4j
public class LoggingAspect {

  @Pointcut("within(com.vendorhub.auth.service..*)")
  public void serviceLayer() {}

  @Around("serviceLayer()")
  public Object logServiceCall(
      ProceedingJoinPoint joinPoint) throws Throwable {

    String methodName =
      joinPoint.getSignature().toShortString();

    log.info("[SERVICE] >> {}", methodName);

    long start = System.currentTimeMillis();

    Object result = joinPoint.proceed();

    long elapsed =
      System.currentTimeMillis() - start;

    log.info("[SERVICE] << {} completed in {}ms",
        methodName, elapsed);
    return result;
  }}
```

Purpose

- Tracks execution flow
- Measures method execution time
- Simplifies debugging and monitoring

7.2.2 Exception Handling Aspect

Global exception handling is implemented using @RestControllerAdvice.

Example: GlobalExceptionHandler

```
@RestControllerAdvice
@Slf4j
public class GlobalExceptionHandler {

  @ExceptionHandler(IllegalArgumentException.class)
  public ResponseEntity<ErrorResponse>
  handleIllegalArgument(IllegalArgumentException ex) {

    return ResponseEntity.status(HttpStatus.BAD_REQUEST)
      .body(new ErrorResponse(
        HttpStatus.BAD_REQUEST.value(),
        ex.getMessage()));
  }
}
```

Purpose

- Centralizes exception handling
- Provides consistent API error responses
- Improves maintainability

7.2.3 Security Cross-Cutting Concern

Authentication and authorization are handled centrally using Spring Security and Gateway Filters.

Example: SecurityConfig

```
@Bean
public PasswordEncoder passwordEncoder() {
  return new BCryptPasswordEncoder();
}
```

Example: JWT Gateway Validation

```
if (!jwtUtil.validateToken(token)) {
  return reject(exchange, HttpStatus.UNAUTHORIZED);
}
```

Purpose

- Centralizes security logic
- Protects all APIs consistently
- Enforces role-based access control

7.2.4 Benefits of AOP in VendorHub

- Reduces duplicated code
- Improves modularity
- Simplifies maintenance
- Separates technical concerns from business logic
- Enhances monitoring and security management

8. Dockerization and Deployment

VendorHub is fully containerized using Docker to simplify deployment, improve portability, and ensure environment consistency across all services. Docker Compose is used to orchestrate the entire microservices ecosystem including backend services, databases, Kafka infrastructure, and the React frontend.

8.1 Docker Architecture

The VendorHub system consists of multiple Docker containers connected through a shared bridge network named vendorhub-net.

The deployed containers include:

- API Gateway
- Eureka Discovery Server
- Config Server
- Authentication Service
- Product Service
- Order Service
- Notification Service
- React Frontend
- Apache Kafka
- Zookeeper
- MySQL databases for each microservice
- MailHog for email testing

8.2 Docker Compose Orchestration

Docker Compose is used to manage and start all services using a single configuration file.

Example: Service Definition

```
auth-service:
  build:
    context: ./auth-service
    dockerfile: Dockerfile
  container_name: auth-service

  depends_on:
    config-server:
      condition: service_healthy
    eureka-server:
      condition: service_healthy
    mysql-auth:
      condition: service_healthy
    kafka:
      condition: service_healthy

  networks:
    - vendorhub-net
```

Purpose

- Automates startup order
- Manages service dependencies
- Simplifies local and cloud deployment
- Reduces manual configuration

8.3 Database Containerization

Each microservice owns an independent MySQL database container following the Database-per-Service microservices pattern.

Example

```
mysql-product:
  image: mysql:8.0
  container_name: mysql-product

  environment:
    MYSQL_ROOT_PASSWORD: rootpassword
    MYSQL_DATABASE: vendorhub_product

  volumes:
    - mysql-product-data:/var/lib/mysql
```

---

9. Microservices and Cloud Architecture

VendorHub follows a microservices-based architecture using Spring Boot and Spring Cloud technologies. The system is divided into independent services that communicate through REST APIs and Apache Kafka event streaming.

This architecture improves scalability, maintainability, fault isolation, and deployment flexibility.

9.1 Microservices Overview

VendorHub is composed of the following microservices:

Service	Responsibility
Auth Service	Authentication, registration, user management
Product Service	Product management, categories, reviews
Order Service	Cart management and order processing
Notification Service	Notifications and email handling

Service	Responsibility
API Gateway	Centralized routing and security
Eureka Server	Service discovery
Config Server	Centralized configuration management

9.2 Eureka Service Discovery

VendorHub uses Spring Cloud Eureka for service registration and discovery.

Registered Services

- auth-service
- product-service
- order-service
- notification-service
- api-gateway

Purpose

- Eliminates hardcoded service addresses
- Enables dynamic service discovery
- Simplifies communication between microservices

9.3 Spring Cloud Config Server

A centralized configuration server is used to manage application configurations for all services.

Purpose

- Centralized configuration management
- Shared environment settings
- Easier updates and maintenance

9.4 API Gateway Architecture

VendorHub uses Spring Cloud Gateway as a single entry point for all client requests.

Responsibilities

- Request routing
- JWT authentication
- Role-based authorization
- Security filtering
- Request forwarding

Example: Authentication Filter

```
if (!jwtUtil.validateToken(token)) {
  return reject(exchange, HttpStatus.UNAUTHORIZED);
}
```

9.5 Event-Driven Communication Using Kafka

VendorHub uses Apache Kafka to enable asynchronous communication between services.

Published Events

- UserRegisteredEvent
- VendorApprovedEvent
- ProductApprovedEvent
- ProductOutOfStockEvent
- OrderPlacedEvent

Example: Kafka Producer

```
kafkaTemplate.send(
  ORDER_PLACED_TOPIC,
  event.getOrderNumber(),
  event);
```

Example: Kafka Consumer

```
@KafkaListener(
   topics = "order-placed",
   groupId = "notification-group"
)
public void onOrderPlaced(OrderPlacedEvent event) {
   notificationService.handleOrderPlaced(...);
}
```

9.6 Database-per-Service Pattern

Each microservice owns a dedicated database instance.

Databases Used

- vendorhub_auth
- vendorhub_product
- vendorhub_order
- vendorhub_notification

9.7 Scalability in VendorHub

VendorHub supports horizontal scalability because services are independently deployable.

Examples

- Product Service can scale separately under heavy browsing traffic
- Notification Service can scale independently during high event loads
- Kafka consumers can process events concurrently

Scalability Features

- Stateless services
- Containerized deployment
- Independent databases
- API Gateway routing
- Kafka event streaming

9.8 Fault Isolation and Reliability

Microservices improve system reliability because failures are isolated within individual services.

Example

- Notification Service failure does not stop order processing
- Kafka queues events until consumers recover
- Independent databases reduce cascading failures

Reliability Mechanisms

- Health checks
- Service isolation
- Kafka asynchronous messaging
- Docker container restart capability

---

10. Clean Code Principles

VendorHub follows several clean code and software engineering principles to improve readability, maintainability, scalability, and code quality across all microservices.

10.1 Separation of Concerns

The application is divided into multiple layers where each layer has a single responsibility.

Layers Used

- Controller Layer
- Service Layer
- Repository Layer
- DTO Layer
- Security Layer

Example

- Controllers handle HTTP requests

- Services contain business logic
- Repositories manage database access

10.2 Layered Architecture

VendorHub follows a layered architecture pattern across all services.

Structure Example

Controller → Service → Repository → Database

Benefits

- Simplifies debugging
- Improves scalability
- Enables independent development
- Supports clean separation between application components

10.3 Meaningful Naming Conventions

Clear and descriptive names are used for:

- Classes
- Methods
- Variables
- APIs
- Events

Examples

- ProductService
- NotificationEventConsumer
- publishOrderPlaced
- findApprovedByPriceRange

10.4 DTO Usage

VendorHub uses Data Transfer Objects (DTOs) to separate API models from database entities.

Example

```
public static class LoginRequest {

   @NotBlank @Email
   private String email;

   @NotBlank
   private String password;
}
```

10.5 Global Exception Handling

VendorHub uses centralized exception handling to maintain consistent API responses.

Example

```
@RestControllerAdvice
public class GlobalExceptionHandler {

   @ExceptionHandler(IllegalArgumentException.class)
   public ResponseEntity<ErrorResponse>
   handleIllegalArgument(IllegalArgumentException ex) {

      return ResponseEntity.status(HttpStatus.BAD_REQUEST)
            .body(new ErrorResponse(
                  HttpStatus.BAD_REQUEST.value(),
                  ex.getMessage()));
   }
}
```

10.6 Validation and Input Checking

Validation annotations are used to validate incoming requests automatically.

Example

```
@NotBlank
@Size(min = 8)
private String password;
```

10.7 Logging and Monitoring

AOP logging is used to monitor application behavior and simplify debugging.

Example

```
log.info("[SERVICE] >> {}", methodName);
```

10.8 Modularity

Each microservice is independently organized and modularized.

Examples

- Separate authentication service
- Separate product service
- Separate notification service

Benefits

- Independent development
- Easier scaling
- Improved maintainability

---

11. Security Implementation

VendorHub implements multiple security mechanisms to protect APIs, user accounts, and inter-service communication.

11.1 JWT Authentication

VendorHub uses JSON Web Tokens (JWT) for stateless authentication.

Authentication Flow

1. User logs in
2. JWT token is generated
3. Client sends token with requests
4. Gateway validates the token
5. Request is forwarded to services

Example

```
String token = jwtUtil.generateToken(user);
```

11.2 API Gateway Security

The API Gateway centrally validates JWT tokens and authorizes requests.

Example

```
if (authHeader == null ||
   !authHeader.startsWith("Bearer ")) {

   return reject(exchange, HttpStatus.UNAUTHORIZED);
}
```

11.3 Role-Based Authorization

VendorHub supports role-based access control (RBAC).

Supported Roles

- ROLE_ADMIN
- ROLE_VENDOR
- ROLE_CUSTOMER

Example

```
if (path.startsWith("/api/admin/")) {
  return "ROLE_ADMIN".equals(role);
}
```

11.4 Password Encryption

Passwords are encrypted using BCrypt hashing.

Example

```
@Bean
public PasswordEncoder passwordEncoder() {
  return new BCryptPasswordEncoder();
}
```

11.5 Validation Security

Input validation is applied to all major request DTOs.

Example

```
@NotBlank
@Email
private String email;
```

11.6 Stateless Session Management

VendorHub disables server-side sessions and uses stateless authentication.

Example

```
.sessionManagement(sm ->
  sm.sessionCreationPolicy(
    SessionCreationPolicy.STATELESS))
```

11.7 Exception Security Handling

Sensitive internal exceptions are hidden from clients using centralized exception handling.

Example

```
return ResponseEntity.status(
  HttpStatus.INTERNAL_SERVER_ERROR)
  .body(new ErrorResponse(
    500,
    "An unexpected error occurred"));
```

---

12. Database Design

VendorHub uses MySQL databases with JPA/Hibernate for data persistence.
The system follows the Database-per-Service pattern where each microservice owns its own database.

12.1 Main Entities

The system includes the following main entities:

Entity	Description
User	Stores customers, vendors, and admins
Product	Stores product information
Category	Stores product categories
ProductReview	Stores customer reviews
Order	Stores customer orders

Entity	Description
OrderItem	Stores products inside orders
Cart	Stores customer cart
CartItem	Stores products inside cart
Favorites	Stores favorite products
Notification	Stores user notifications

12.2 User Entity

The User entity manages authentication and authorization information.

Main Attributes

- id
- email
- password
- role
- status
- approvalStatus
- businessName

Supported Roles

- Admin
- Vendor
- Customer

12.3 Product Entity

The Product entity stores product-related information.

Main Attributes

- name
- description
- price
- stockQuantity
- category
- vendorId
- approvalStatus
- averageRating

Features

- Product approval workflow
- Stock tracking
- Product reviews
- Product categorization

12.4 Order Management Entities

The ordering system consists of:

- Order
- OrderItem
- Cart
- CartItem

Features

- Cart management
- Checkout processing
- Order tracking
- Vendor sales updates

12.5 Notification Entity

The Notification entity stores:

- Vendor notifications
- Product approval notifications
- Low stock alerts
- Order notifications

Notifications are generated asynchronously using Kafka events.

12.6 Database Relationships

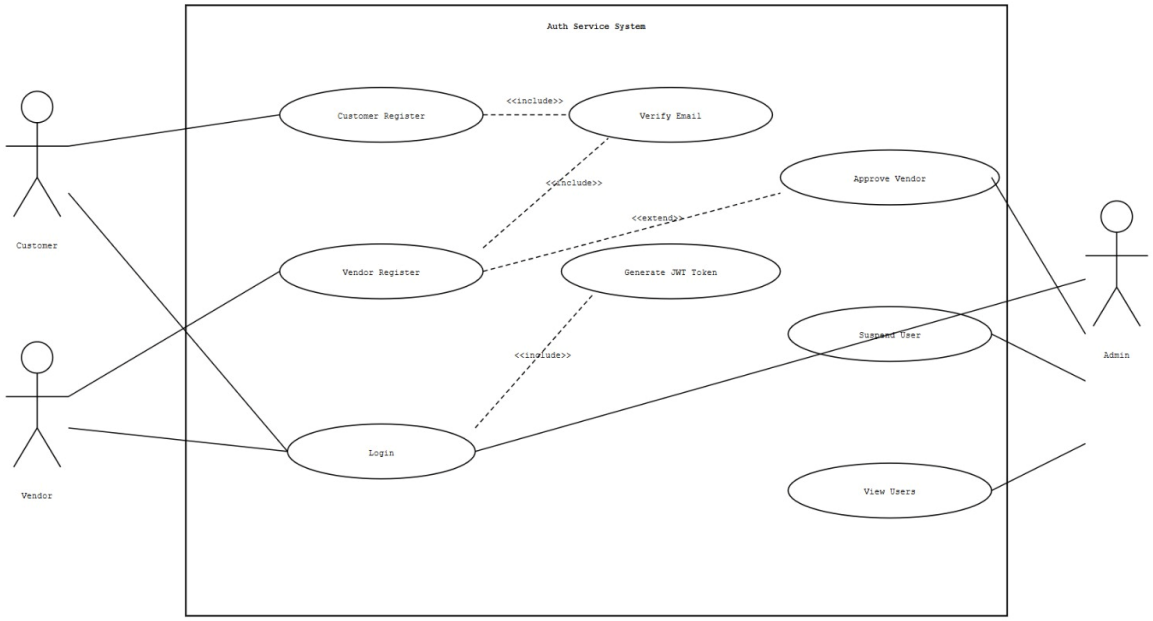
Main Relationships

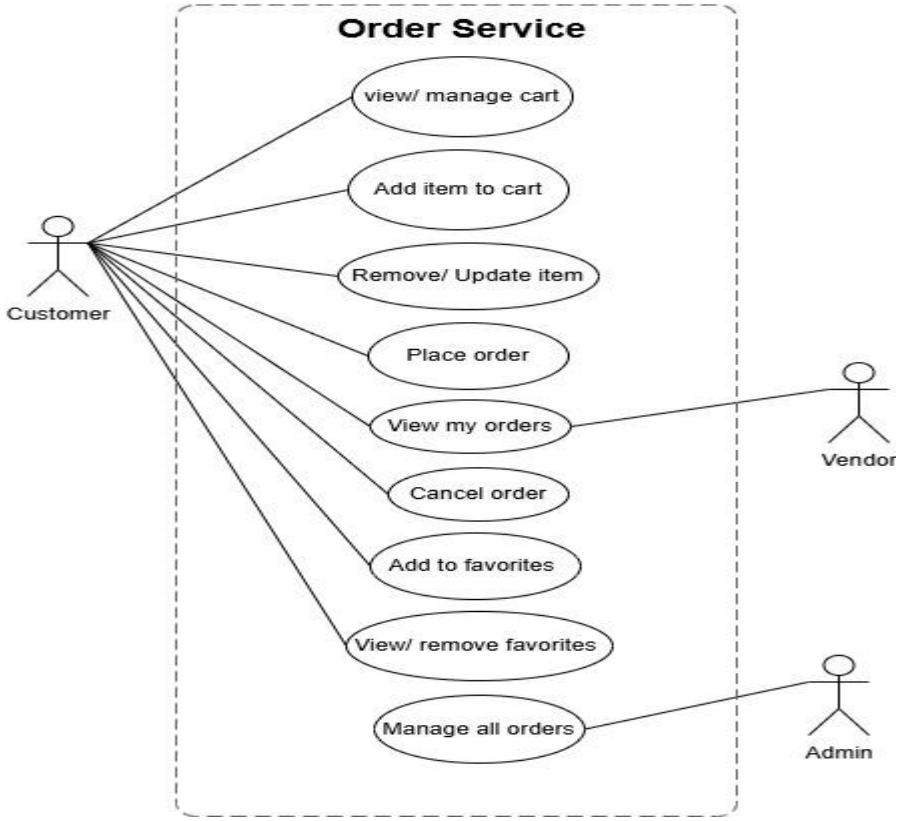
- One Vendor → Many Products
- One Category → Many Products
- One Product → Many Reviews
- One Order → Many OrderItems
- One Customer → One Cart
- One Cart → Many CartItems

---

13 System Diagrams

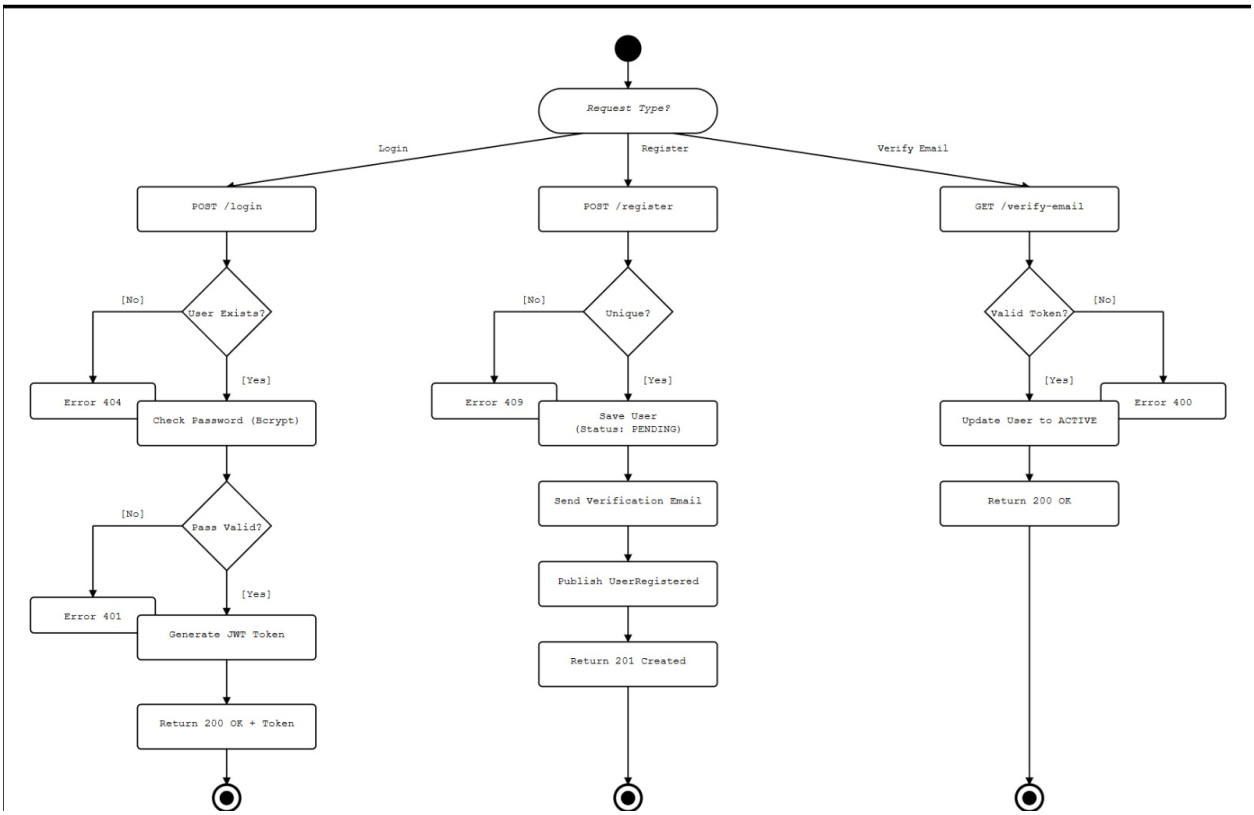
13.1 Use Case Diagrams





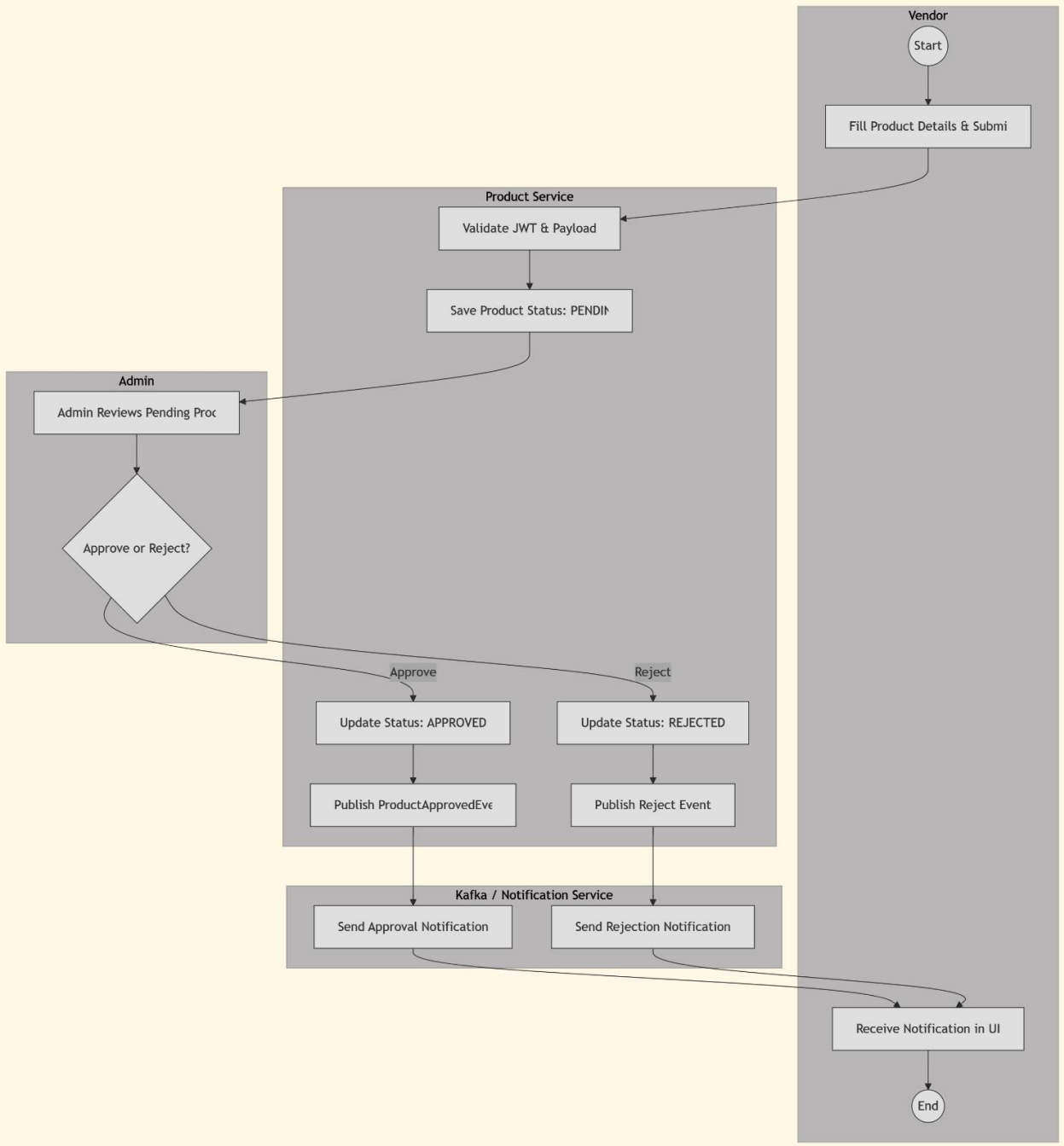
13.2 Activity Diagrams

13.2.1 Auth Service

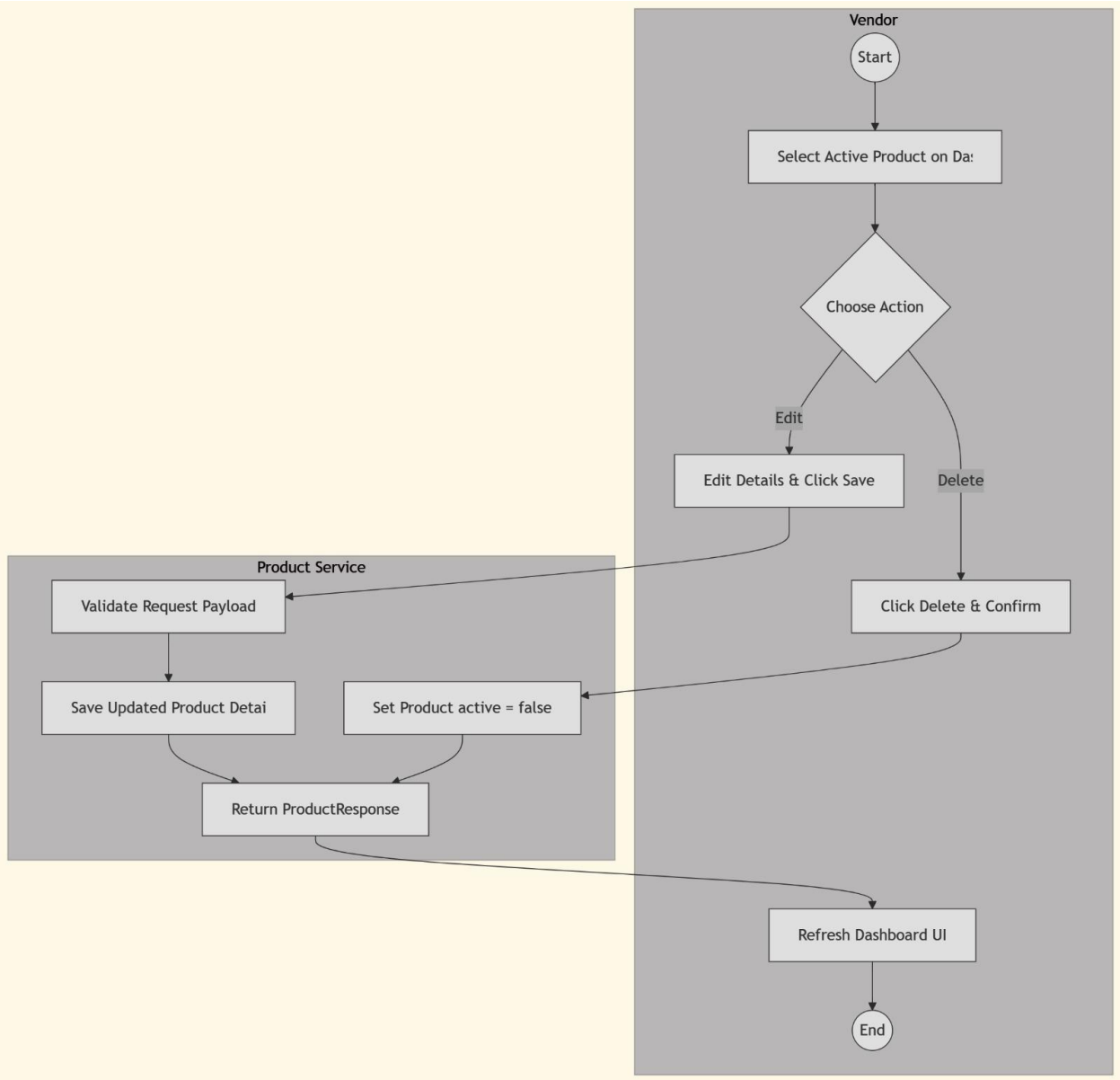


13.2.2 Product Service

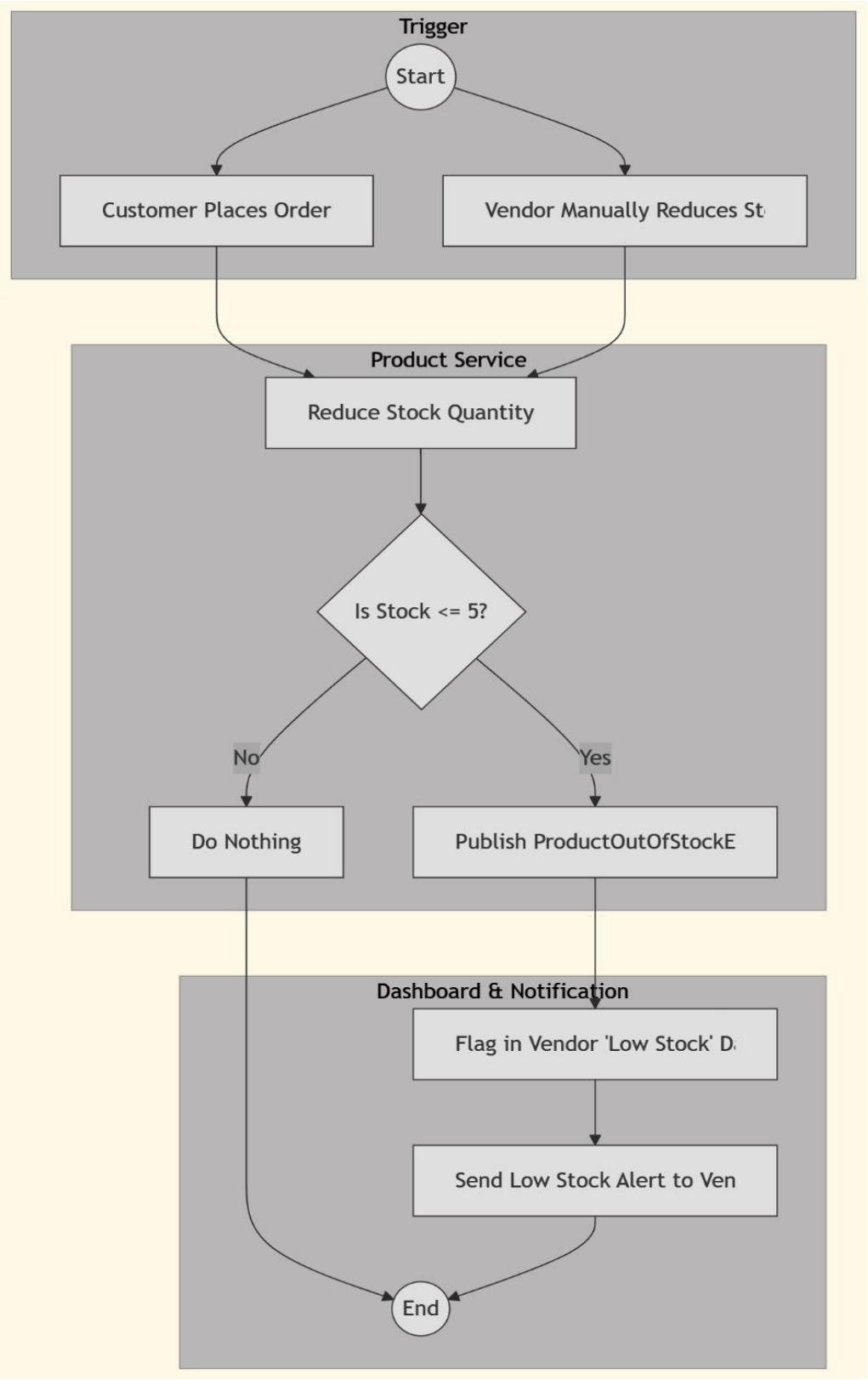
Creation & Approval Workflow



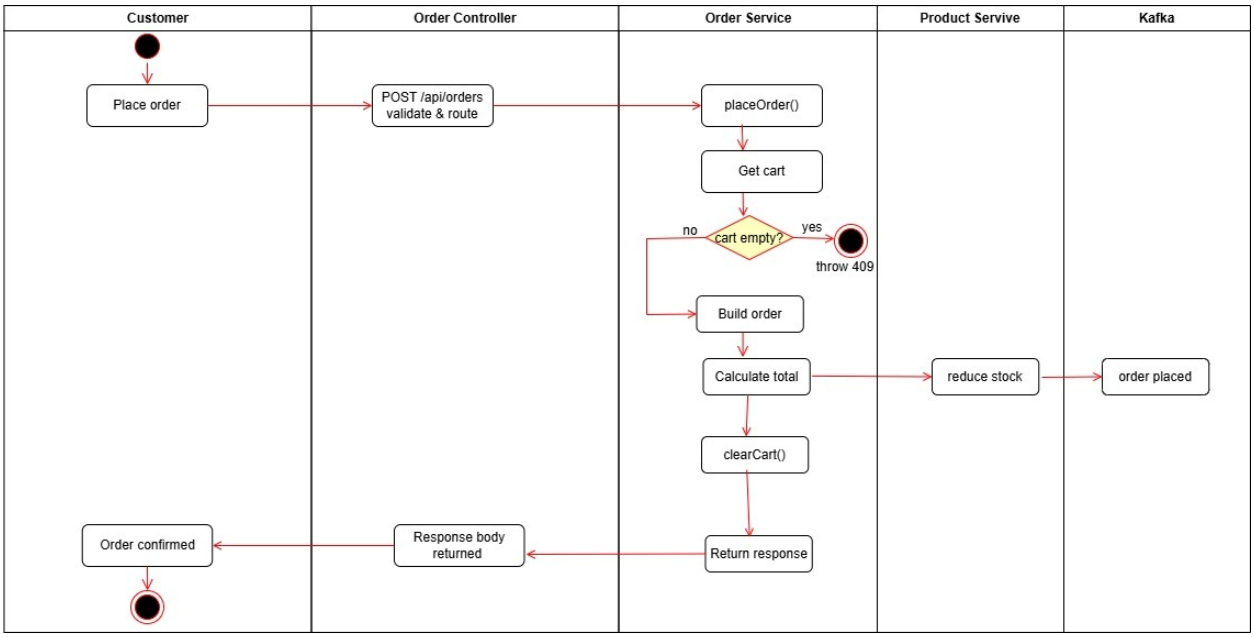
Vendor Product Management (Update & Soft Delete)



The Low Stock Alert Flow

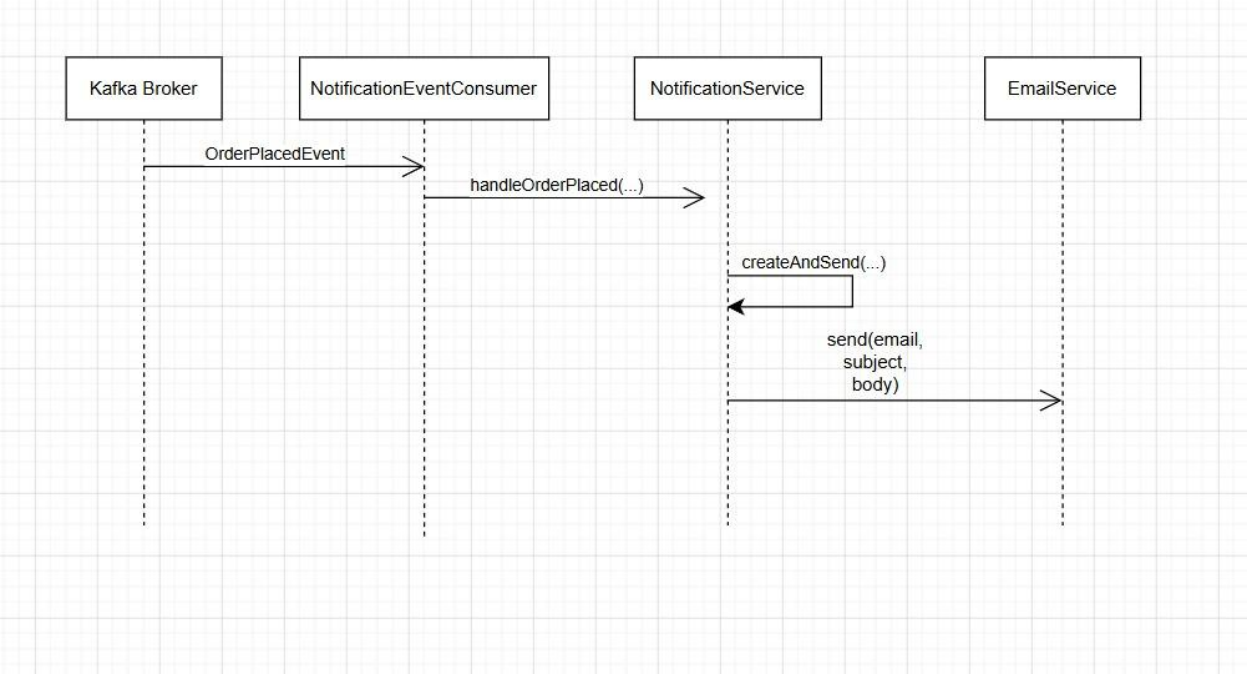


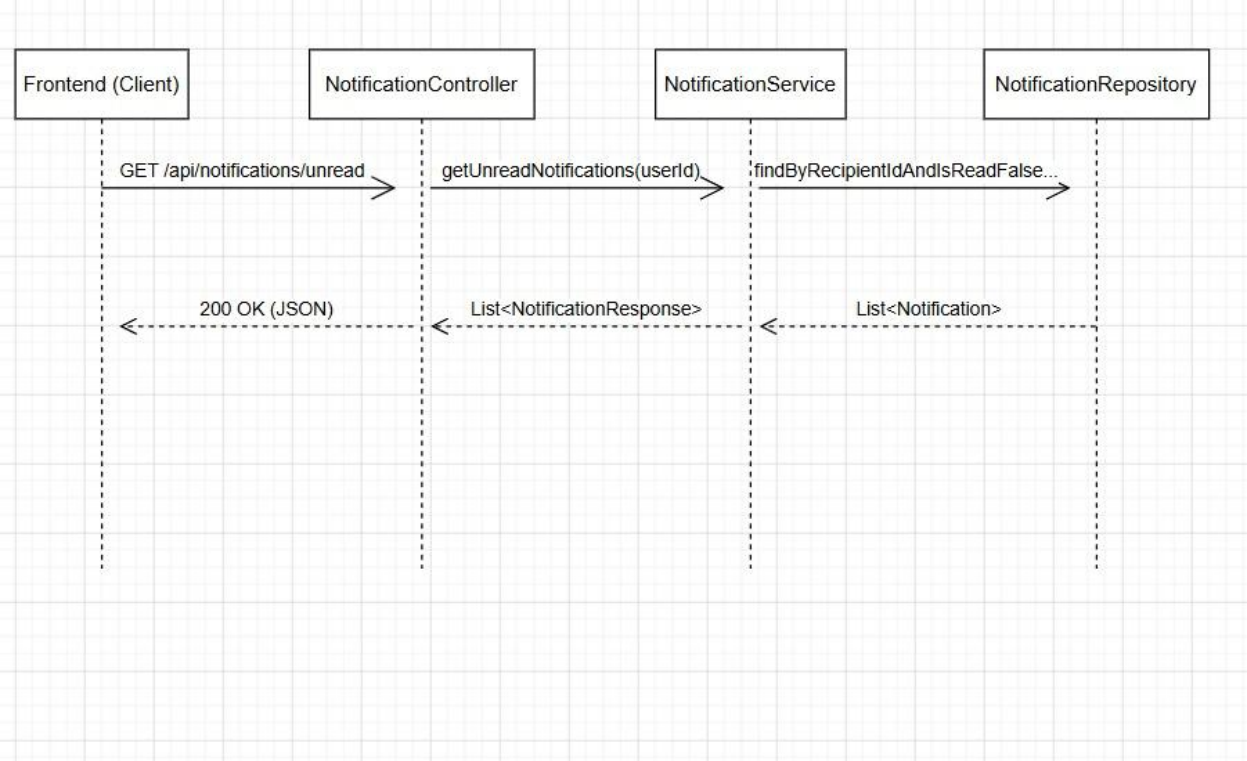
13.2.3 Order Service



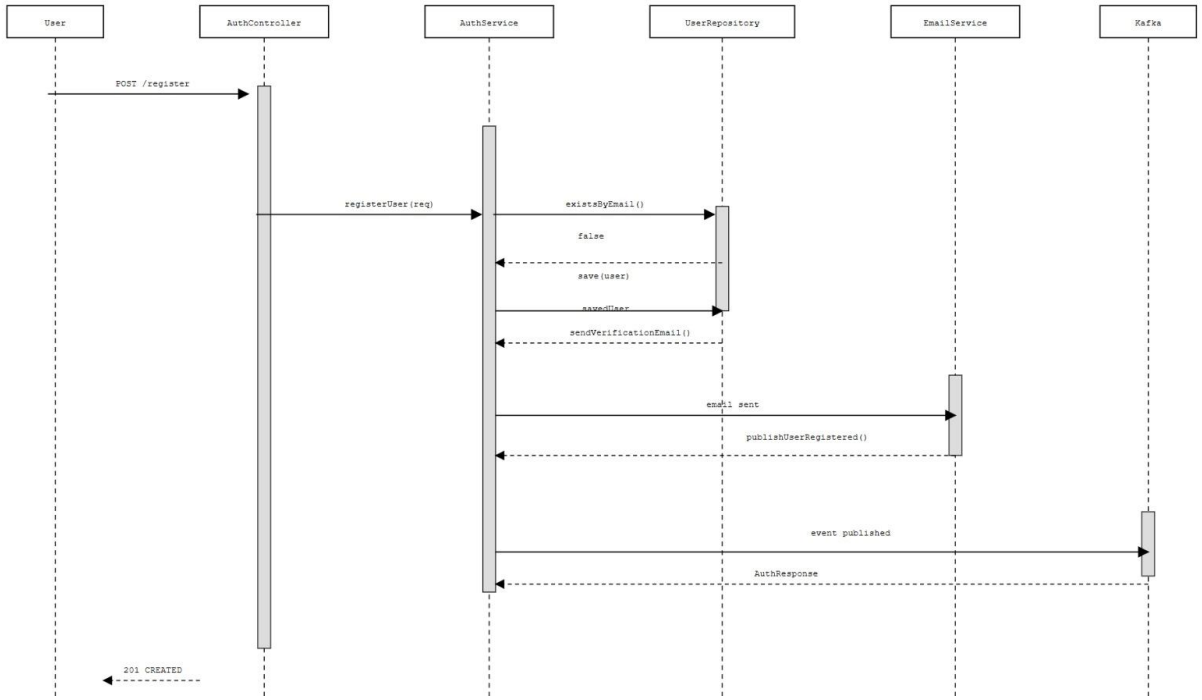
13.3 Sequence Diagrams

13.3.1 Notification Service



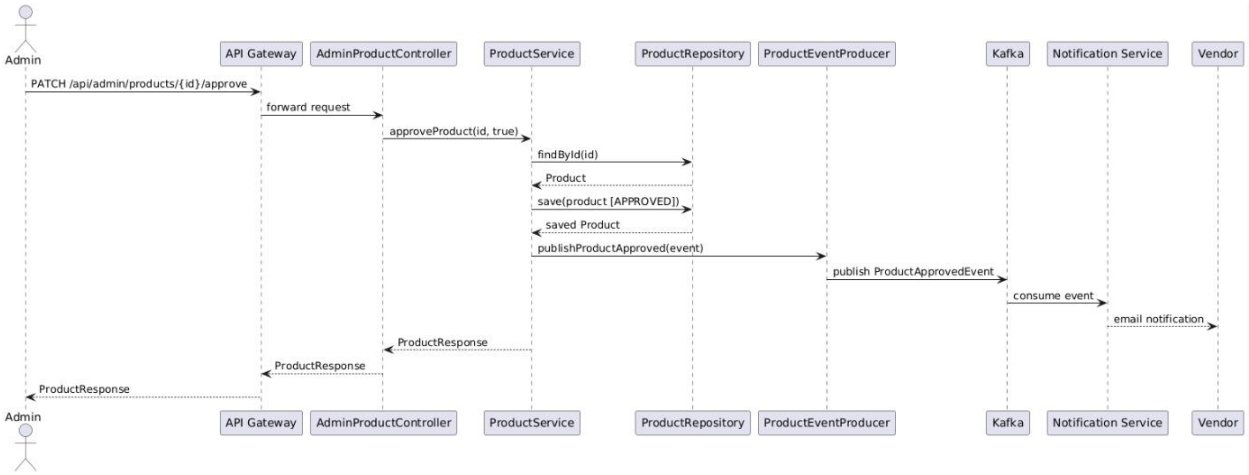


13.3.2 Auth Service

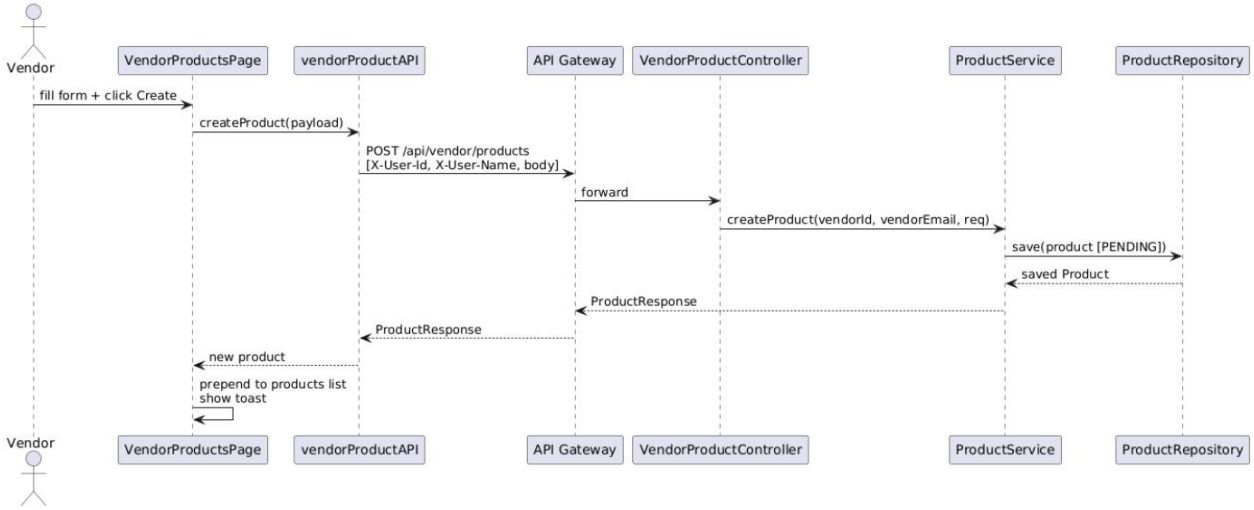


13.3.3 Product Service

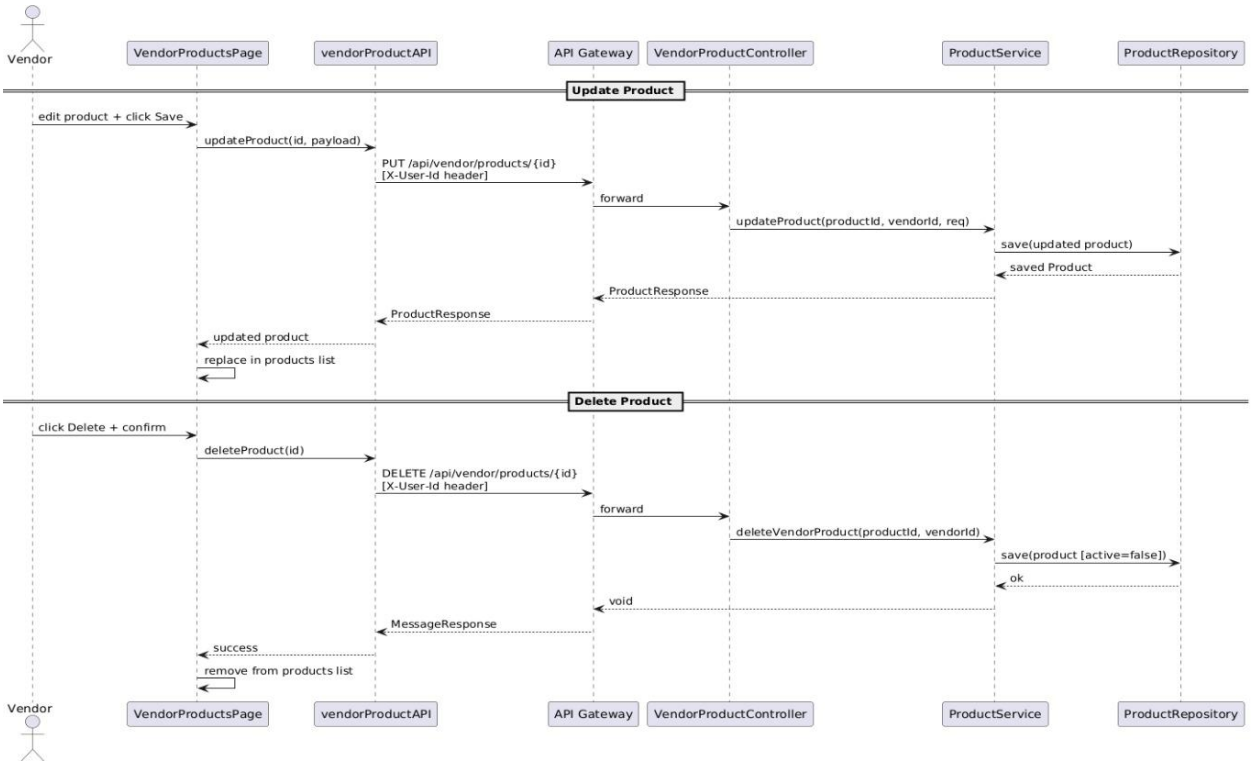
Admin Approves a Product (with Kafka)



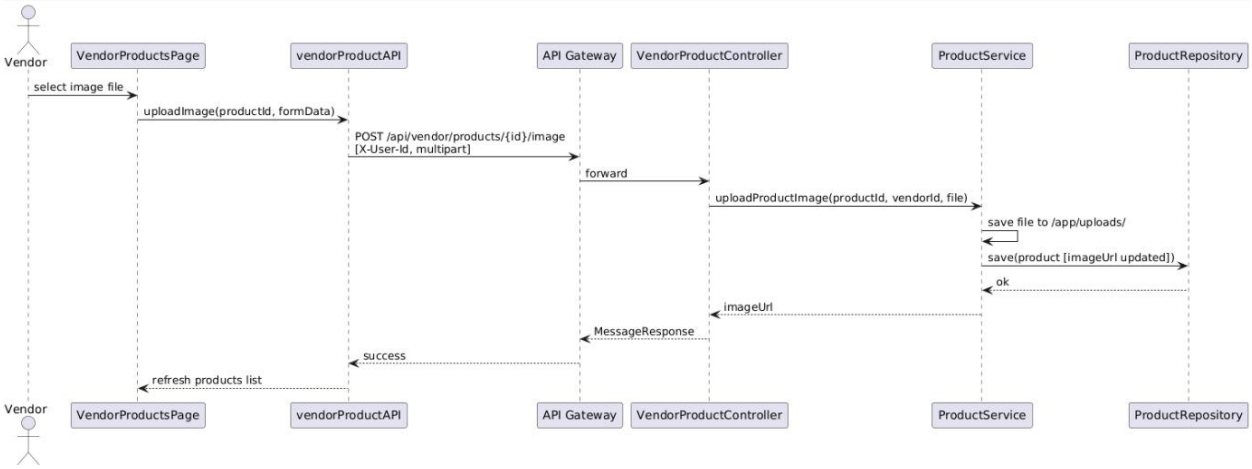
create product



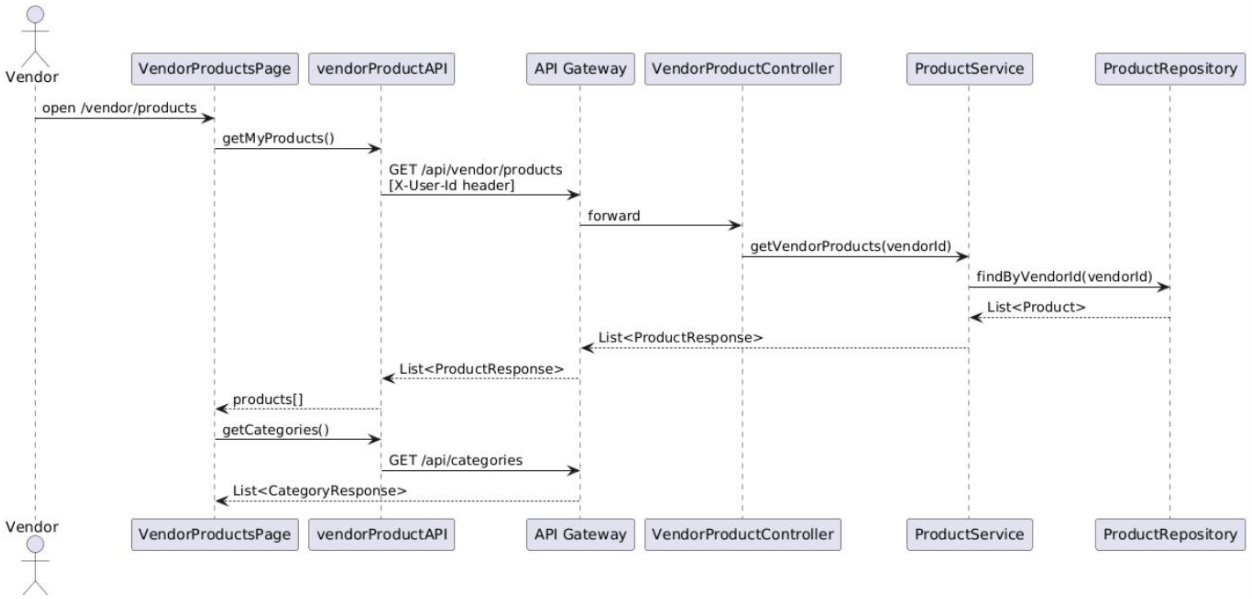
Update & delete product



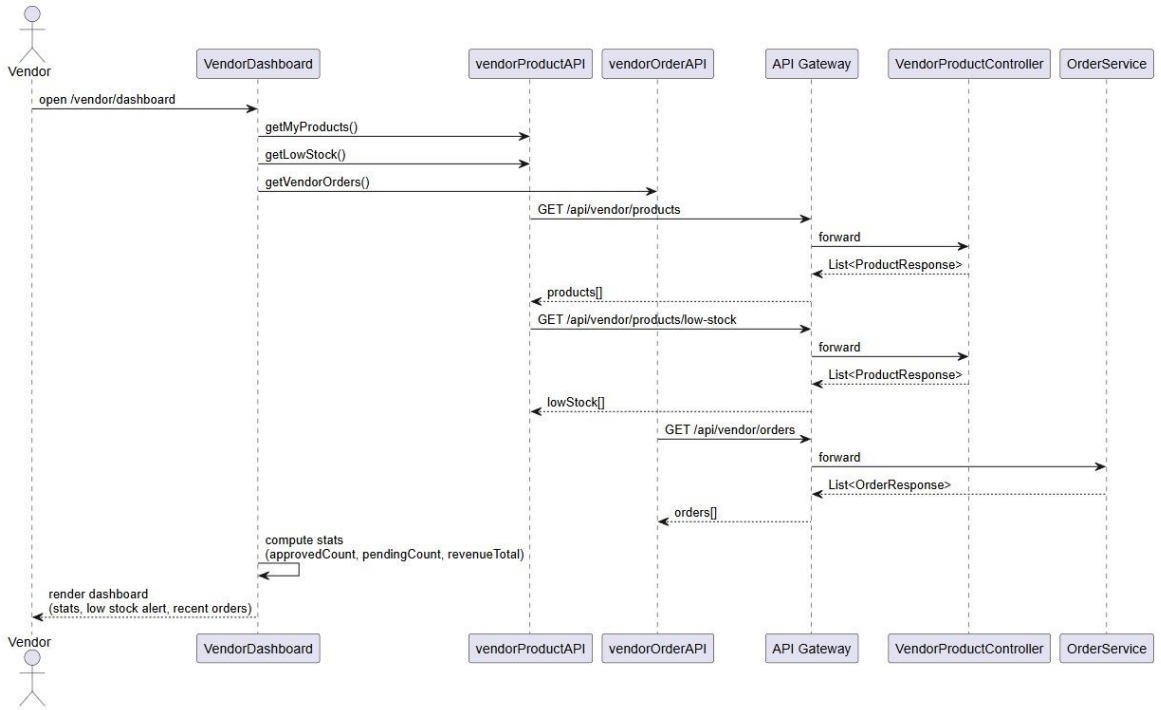
Upload image



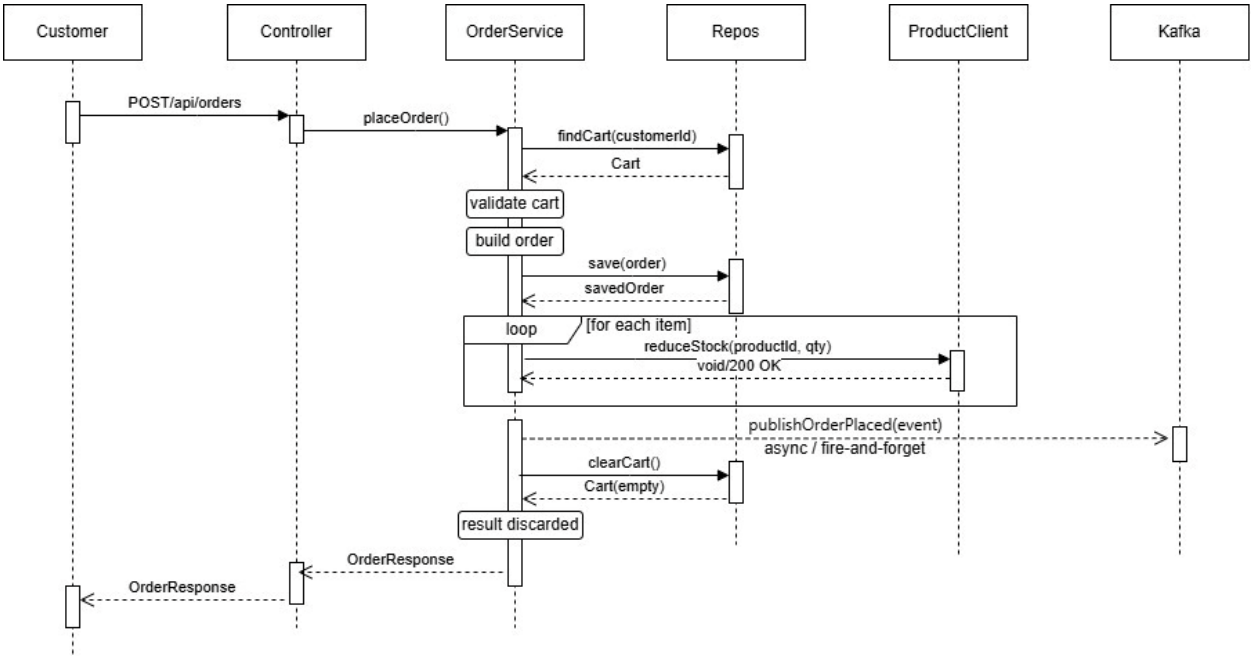
Load Vendor's Products Page



VendorDashboard

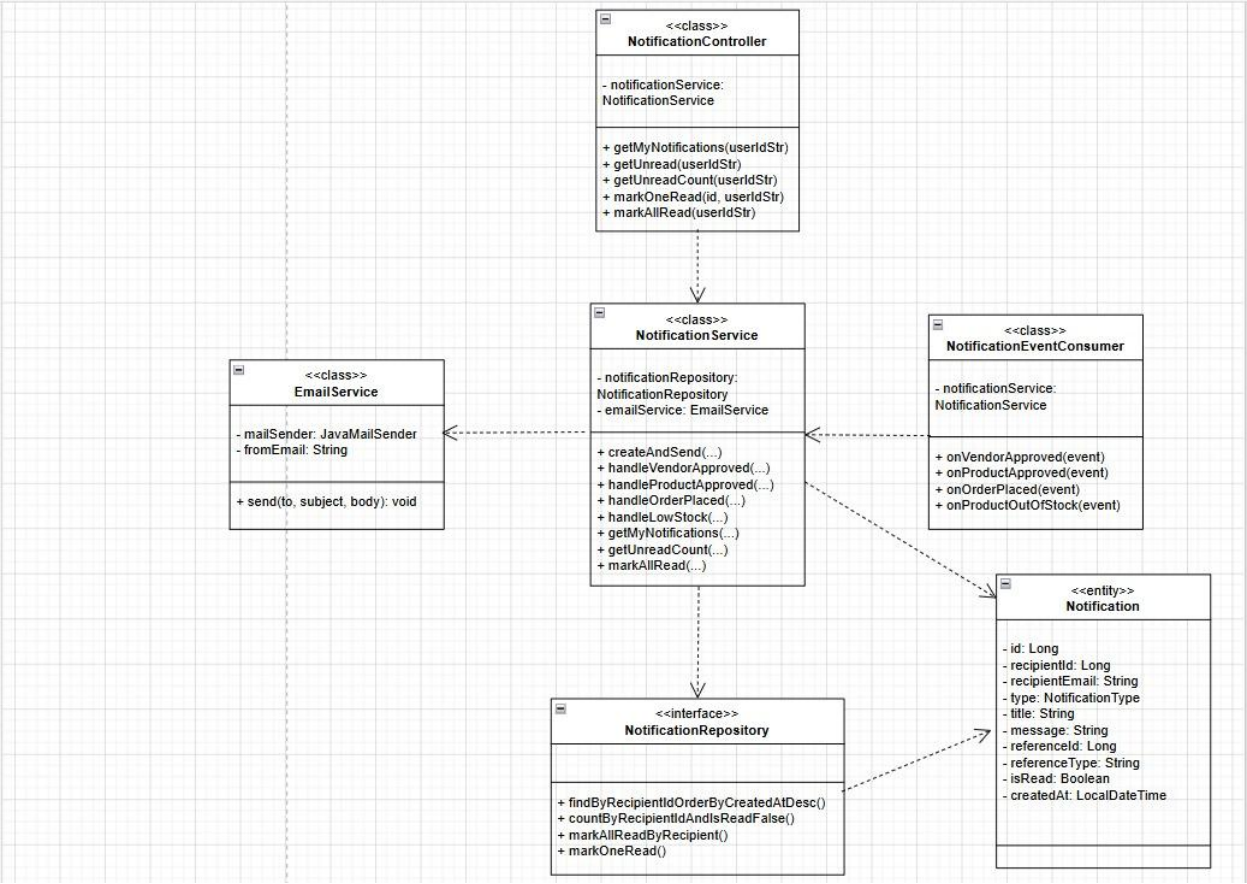


13.3.4 Order Service

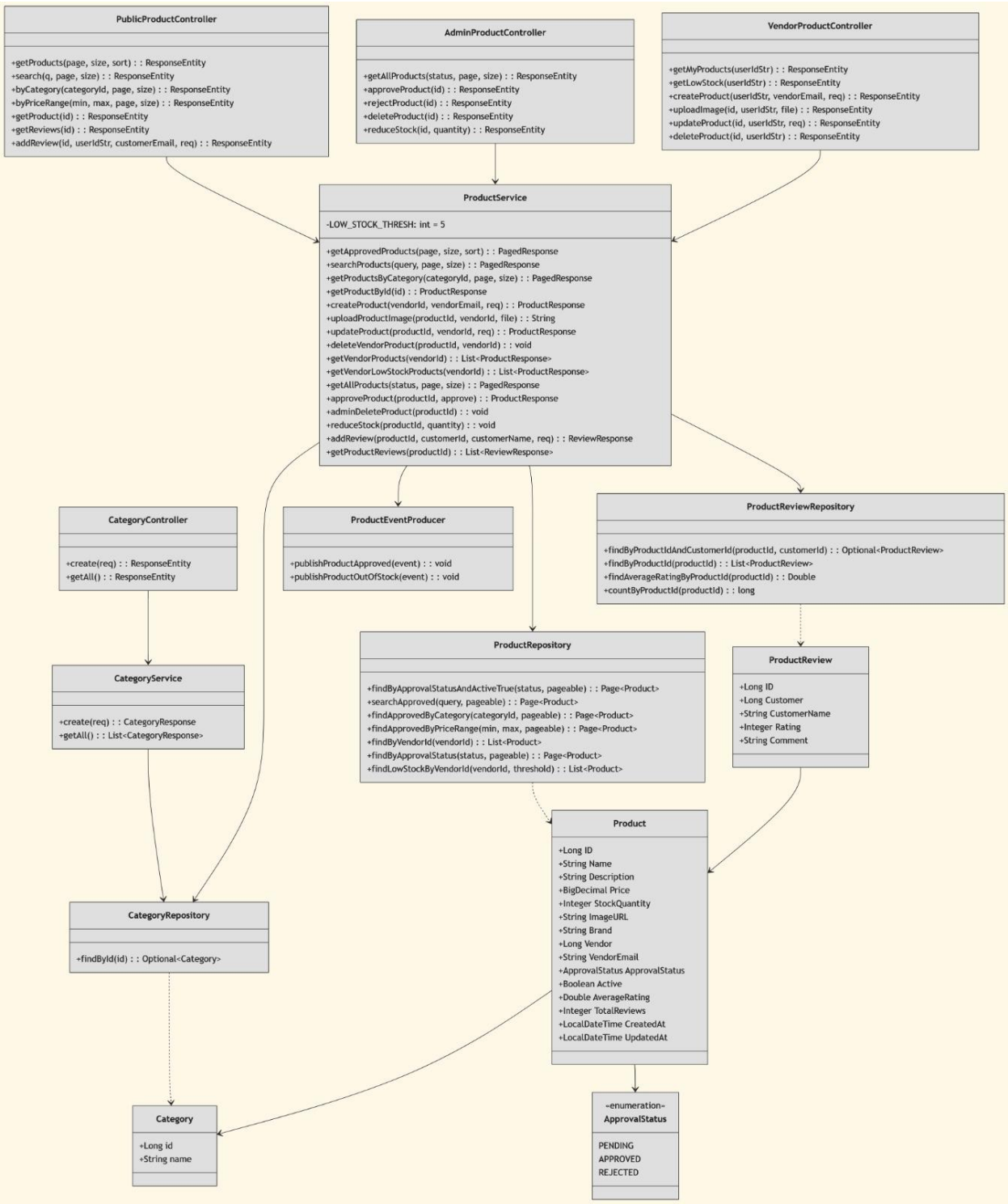


13.4 Class Diagram

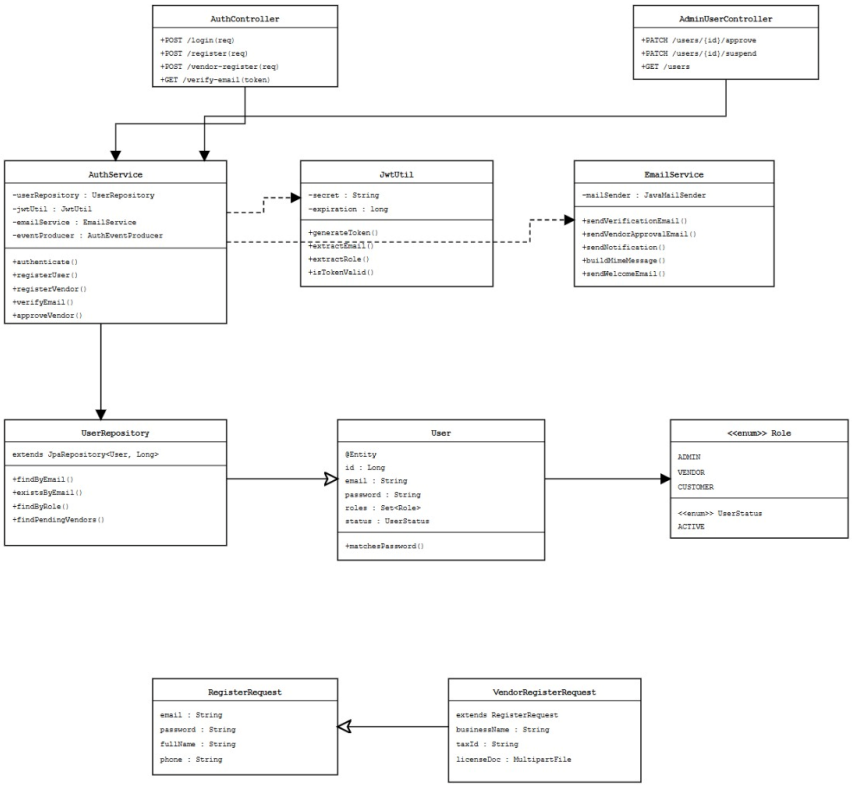
13.4.1 Notification Service



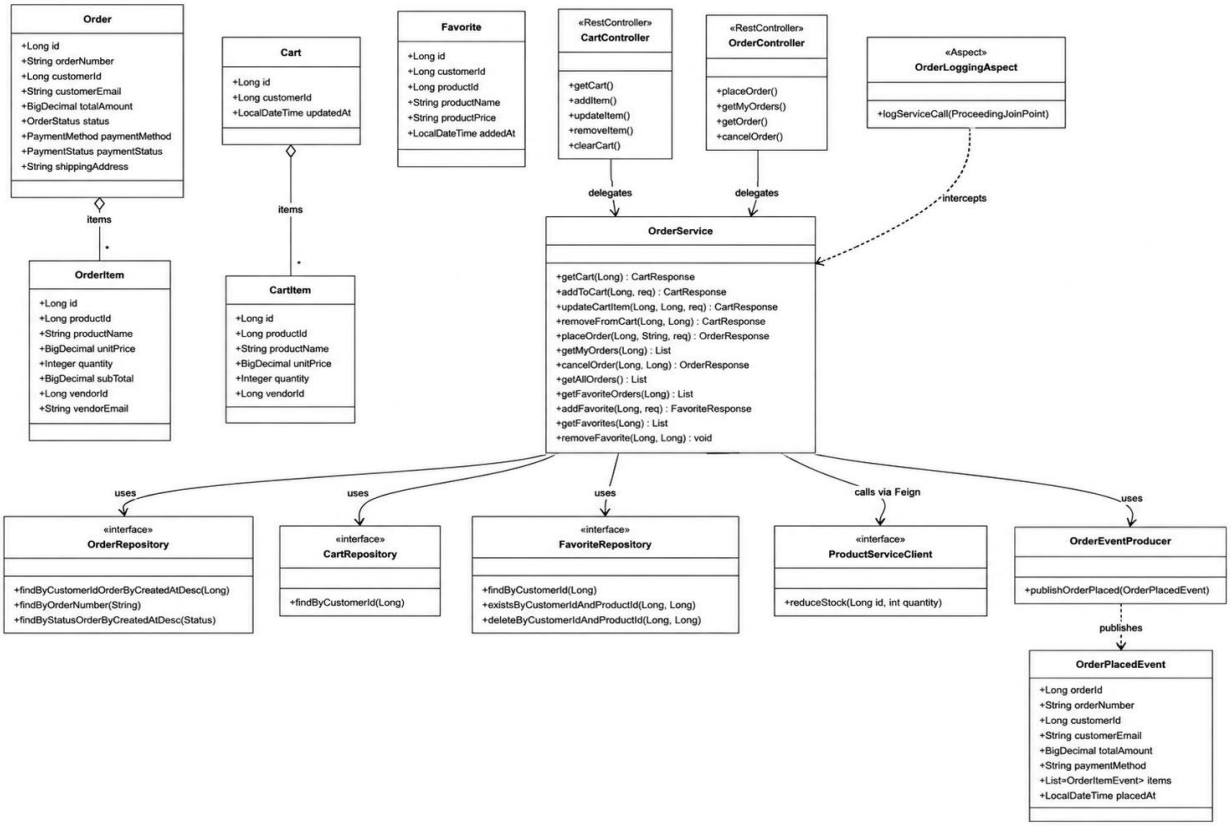
13.4.2 Product Service



13.4.3 Auth Service

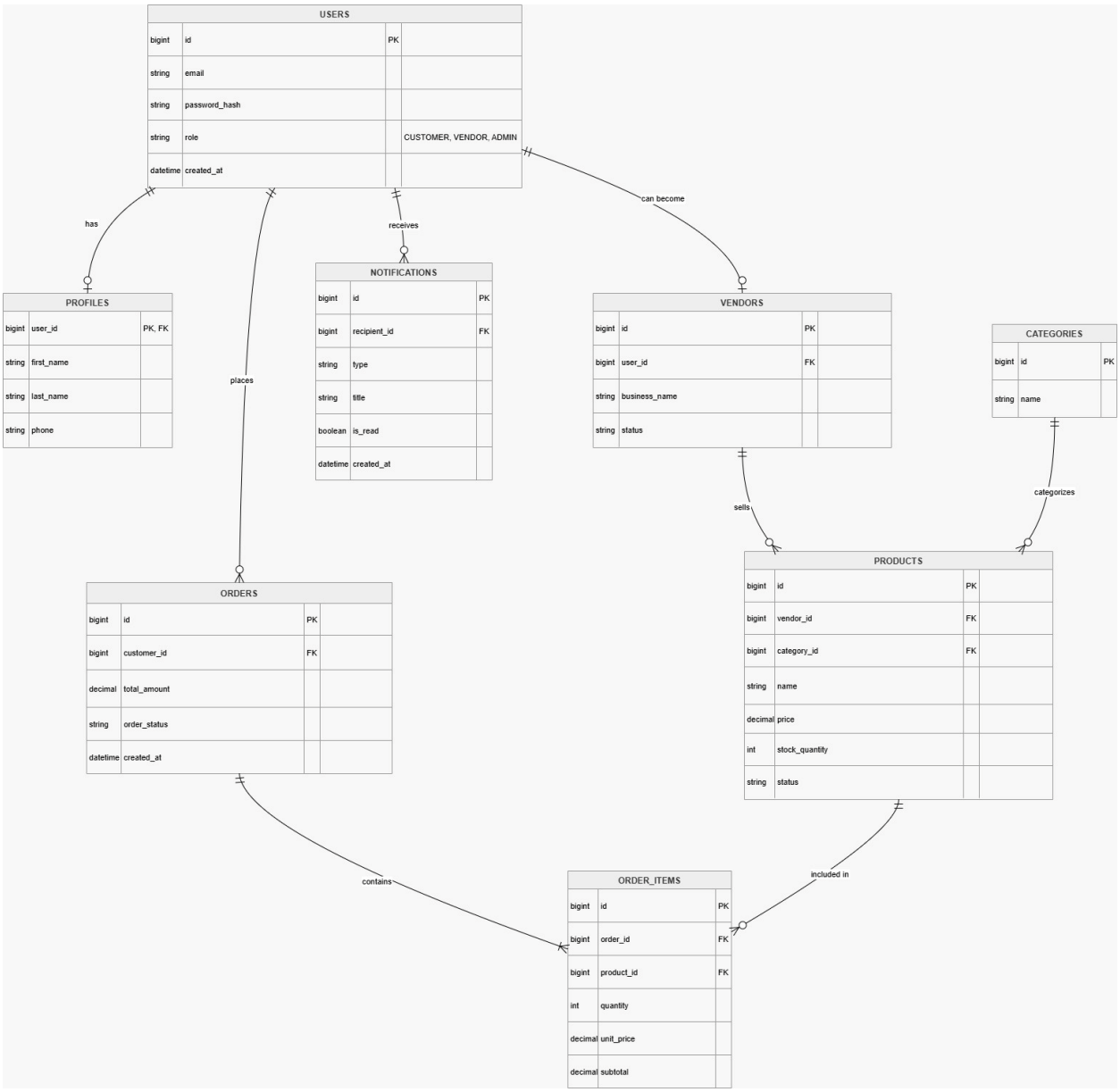


13.4.4 Order Service

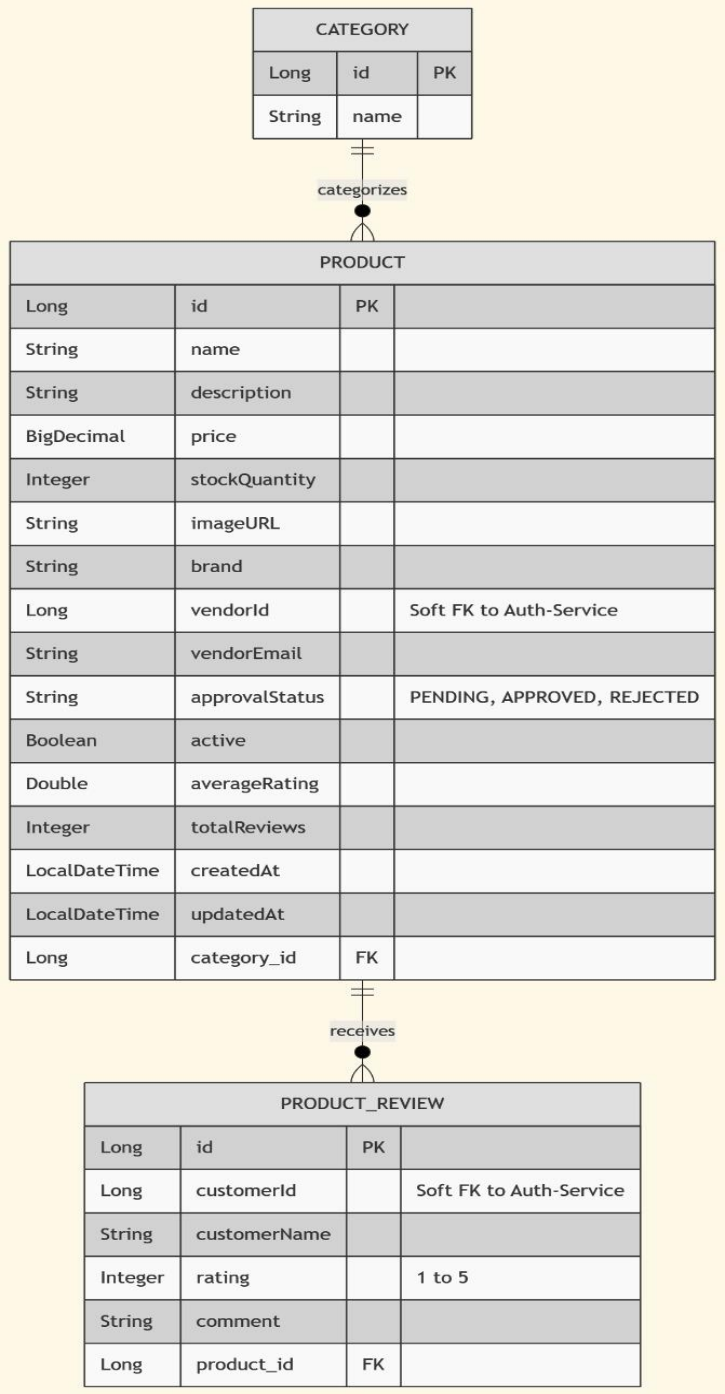


13.5 Entity Relationship Diagram (ERD)

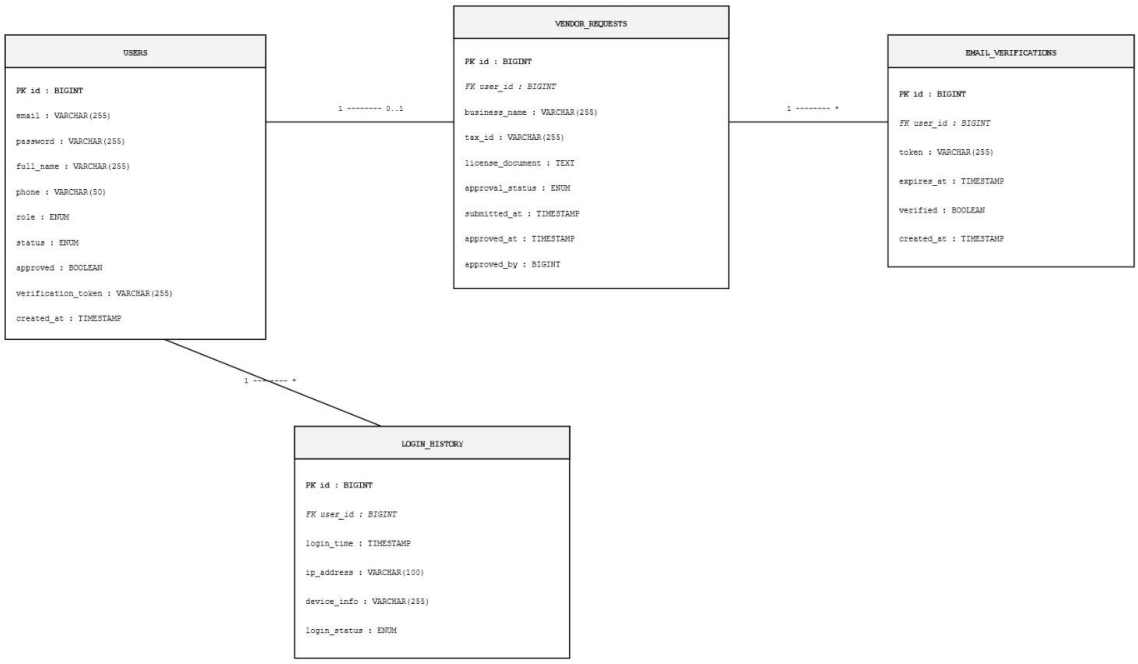
13.5.1 Notification Service



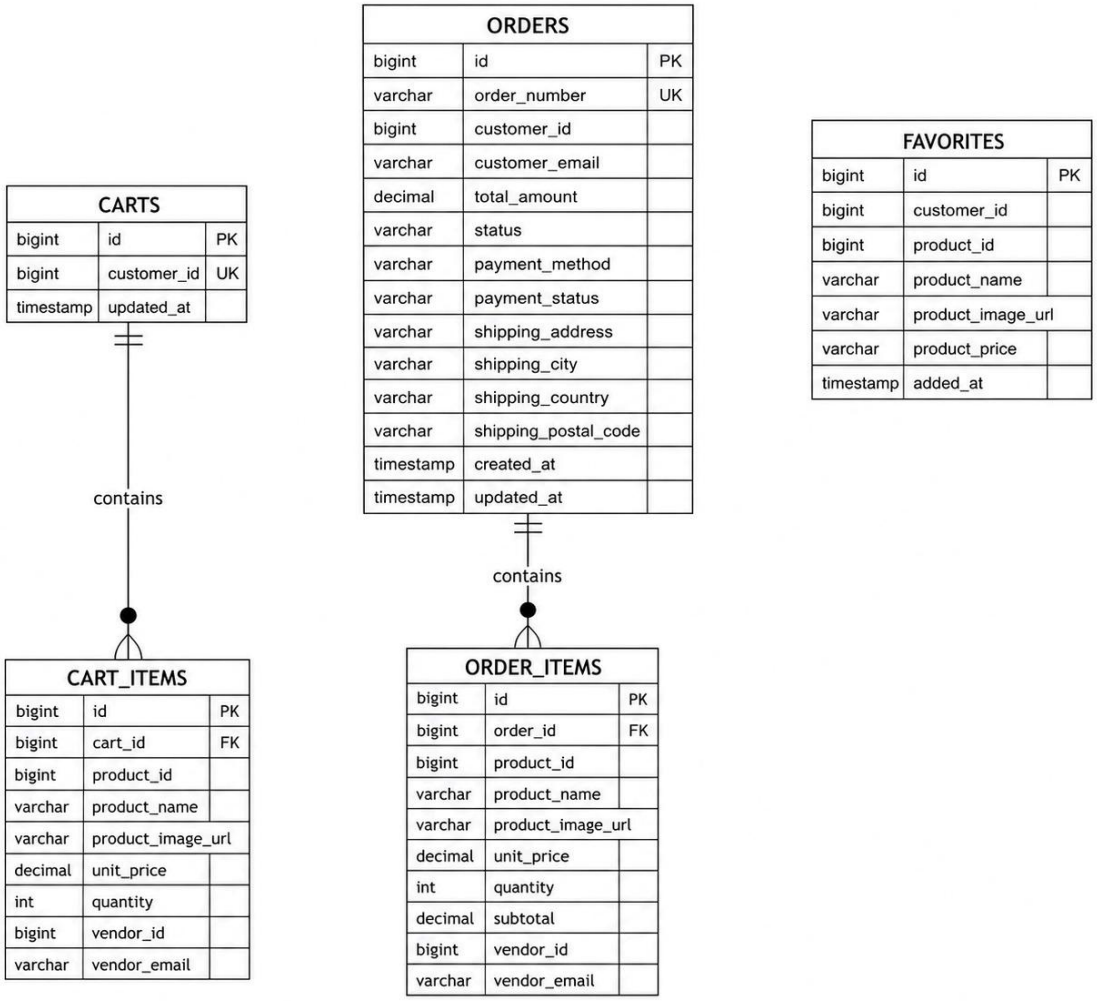
13.5.2 Product Service



13.5.3 Auth Service



13.5.4 Order Service



14. Conclusion

VendorHub is a scalable microservices-based e-commerce platform designed to support administrators, vendors, and customers through a modular and maintainable architecture.

The project successfully integrates:

- Spring Boot
- Spring Cloud
- REST APIs
- Docker
- Apache Kafka
- JWT Authentication
- React Frontend
- MySQL Databases

The system implements modern software engineering concepts including:

- Microservices architecture
- Event-driven communication
- Aspect-Oriented Programming
- Repository and Service Layer patterns
- Centralized security
- Dockerized deployment

VendorHub achieves:

- Scalability
- Maintainability
- Security
- Reliability
- Modular deployment

The use of Docker, Kafka, Spring Cloud, and independent services makes the platform cloud-ready and suitable for future expansion and distributed deployment environments.